

What is Fetch API

→ JS interface used for making HTTP request to servers.

→ Return Promises

→ Used to get data

QS

Basic Method / then.catch (Promise chain)

```
fetch('https://api.example.com/data')
```

```
  .then(response => response.json())
```

```
  .then(data => console.log(data))
```

```
  .catch(error => console.error('Error', error))
```

Modern Method - async/await

QS

```
async function fetchData() {
```

```
  try {
```



```

const response = await fetch(.....);
const data = await response.json();
console.log(data);
} catch (error) {
  console.error('Error', error);
}
}

```

Handling Loading, Success, Error State

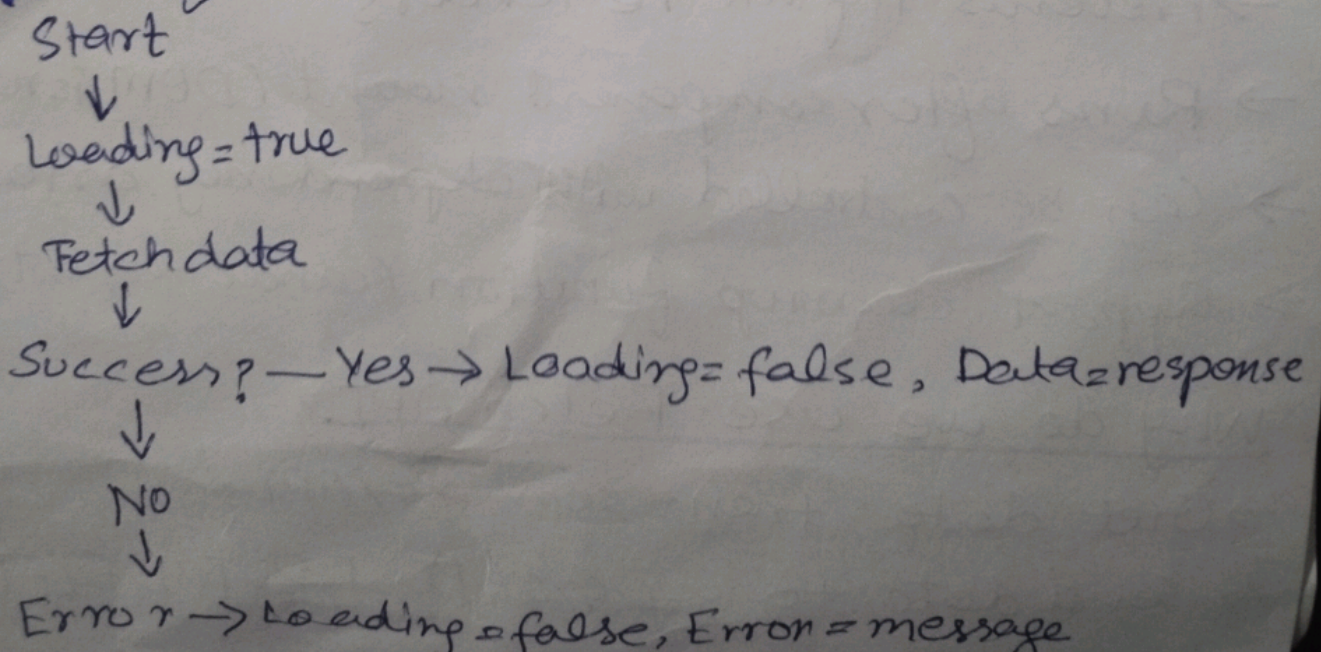
→ In react we track three states when fetching data:

```

const [data, setData] = useState(null);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

```

Flow 41



- Don't call fetch in component body Q1
- Call it in useEffect

```
function Component() {  
  const [data, setData] = useState(null);  
  useEffect(() => {  
    fetch(-----)  
    .then(res => res.json())  
    .then(data => setData(data));  
  }, []);  
}
```

Why useEffect? Q3

- Prevents infinit re-renders
- Runs after component mount (DOM is ready)
- Can be controlled with dependency array
- Support cleanup function (cancel request)

Why do we use Fetch API?

- Get data from servers/APIs
- Send data to servers (POST, PUT, DELETE)

- Update React App with external data
- Create dynamic, data, driven apps

Fetch

- Built in Javascript
- Manual JSON parsing required
- No request cancellation
- Less browser support
- Lightweight

`fetch(url)`

- `then(res => res.json())`
- `then(data => setData(data));`

Axios

- External library
- Auto JSON parsing
- Built-in request ~~cancel~~ cancellation
- Better Browser support
- More features, slightly heavier

`axios.get(url)`

- `then(res => console.log(res.data) console.log(res.data));`

What happens if you forget the dependency array in useEffect when fetching data

Fetch will run on every render, causing

- Infinite loop (fetch → update state → re-render → fetch)
- Performance issues
- App crash
- Wasted API calls

Always use dependency Array

- run once on mount - `[]`
- run when variable change - `[variable]`